API SPECIFICATION DOCUMENT

Project Name: DevExplorer

Implementation: HybridADT Interface

Author: Asad Ullah (3482-2024)

Version: 2.0

1. Executive Summary

This document specifies the Abstract Data Type (ADT) interface for the DevExplorer backend. The API follows a "Hybrid" design, combining multiple classical data structures (Trees, Tries, and Inverted Indices) to provide high-performance file operations. The API is designed to be platform-independent and UI-agnostic.

2. Global Exception & Error Handling

To ensure the "Fail-Safe Behavior" mentioned in the project report, all methods implement the following error handling strategy:

Exception Type	Triggering Condition	API Recovery Behavior
PathNotFoundException	Invalid or moved directory paths.	Logs error; maintains current state.
AccessDeniedException	System/Hidden folders (e.g., System Volume Information).	Gracefully skips the entry during indexing.
Indexing Exception	Corruption during index building.	Reverts to last successful persisted state.
ConcurrentException	Violation of the two-thread execution model.	Synchronizes operation or blocks till resource is free.

3. Data Model Definitions

These models are used as return types for the API methods.

3.1 FileMetadata (Record)

- String path: Absolute system path.
- boolean isDirectory: True if the object is a folder.
- long size: File size in bytes.
- String extension: File type (e.g., ".java").

3.2 FileEvent (Record)

- EventType type: CREATE | MODIFY | DELETE.
- Path path: The affected filesystem resource.

4. Method Specifications

4.1 Navigation API

Defines how the ADT models and navigates the filesystem hierarchy.

Method	Parameters	Return Type	Description
`openDirectory`	`String path`	`void`	Changes the current context to the specified path.
`MapsToParent`	`none`	`void`	Moves one level up in the Directory Tree.
`listCurrentDirectory`	`none`	`List<FileMetadata>`	Returns all immediate children of the current path.
`getCurrentPath`	`none`	`String`	Returns the absolute path of the current directory.

4.2 Search API

Treats search as a navigation primitive using high-efficiency data structures.

Method	Data Structure	Complexity	Description
`searchByFilenamePrefix`	**Trie**	$O(k)$	Instant lookup based on filename prefix.
`searchByFileContent`	**Inverted Index**	$O(1)$ lookup	Finds files containing specific text tokens.
`filterByExtension`	**Hash Map**	$O(n)$	Filters the current view by file type.
`rankSearchResults`	**Priority Queue**	$O(n \log n)$	Ranks results based on Term Frequency (TF).

4.3 Indexing & Persistence API

Manages the lifecycle of the hybrid index and disk serialization.

Method	Description	Persistence Context
`buildInitialIndex`	Full crawl using Iterative Deepening DFS.	Cold Start.
`updateIndexOnFileCreate`	Incremental update of Trie and Inverted Index.	Real-time.
`persistIndexToDisk`	Serializes the ADT state to a `.dat` file.	On Shutdown.
`loadIndexFromDisk`	Deserializes state for instant application launch.	On Startup.

4.4 Developer Utility API

Convenience methods for developer-centric workflows.

- openTerminalHere(): Invokes cmd.exe or powershell.exe at the current path.
- toggleCodeMode(boolean): When enabled, the API automatically applies .ignore rules.
- applyIgnoreRules(List<String> patterns): Filters out node_modules, .git, and build artifacts from indices.

5. System Concurrency Contract

The DevExplorer API strictly enforces a **Two-Thread Limit**:

1. **Main/UI Thread:** Responsible for Navigation and Search query execution.
2. **Background Indexer Thread:** Responsible for building and updating indices.

Note: Any method modifying the Index (e.g., updateIndexOnFileModify) is internally synchronized to prevent race conditions during search operations.